

Tugas Analisis Multimedia: Image (Citra Digital)

Mata Kuliah: Sistem & Teknologi Multimedia

Nama: Rizki Alfariz Ramadhan **NIM:** 122140061

Deskripsi Tugas

Tugas ini bertujuan untuk memahami **representasi dasar data citra digital (image)** melalui praktik langsung memuat data, visualisasi komponen warna, serta melakukan analisis spasial sederhana menggunakan berbagai teknik dasar pengolahan citra.

Anda akan bekerja dengan satu atau beberapa gambar (foto diri, objek, atau lingkungan sekitar) untuk:

- Mengamati struktur data piksel dan channel warna (RGB, Grayscale, HSV, dsb.)
- Menganalisis perbedaan hasil visualisasi antar representasi warna
- Melakukan eksplorasi sederhana terhadap transformasi citra (cropping, filtering, edge detection, dll.)
- Menyimpulkan pengaruh setiap tahap pemrosesan terhadap persepsi visual

Fokus tugas ini adalah pada **pemahaman konsep representasi spasial citra digital** dan **interpretasi hasil visualisasi**, bukan pada manipulasi kompleks atau penerapan model pembelajaran mesin.

Soal 1 — Cropping dan Konversi Warna

- Ambil sebuah gambar diri Anda (*selfie*) menggunakan kamera atau smartphone.
- Lakukan **cropping secara manual** untuk menghasilkan dua potongan:
 - Cropping **kotak persegi pada area wajah**.
 - Cropping **persegi panjang pada area latar belakang**.
- Resize hasil crop menjadi **920×920 piksel**.
- Konversi gambar menjadi **grayscale** dan **HSV**, lalu tampilkan ketiganya berdampingan.
- Tambahkan **anotasi teks** berisi nama Anda di atas kepala pada gambar hasil crop.
 - Gaya teks (font, warna, posisi, ukuran, ketebalan) **dibebaskan**.
- Jelaskan efek **cropping** dan **perubahan warna** menggunakan **Markdown**.

Import Library

```
In [2]: import cv2
import os
import matplotlib.pyplot as plt
import numpy as np
```

Import Image

```
In [3]: BASE_DIR = os.getcwd()
IMAGE_PATH = os.path.join(BASE_DIR, 'assets_ws4', 'selfie.jpg')
RESULTS_DIR = os.path.join(BASE_DIR, 'results_ws4')

if not os.path.exists(RESULTS_DIR):
    os.makedirs(RESULTS_DIR)
```

Load Image

```
In [4]: img_bgr = cv2.imread(IMAGE_PATH)

if img_bgr is None:
    print(f"Error: Tidak dapat memuat gambar dari {IMAGE_PATH}")
else:
    print(f"Gambar berhasil dimuat, ukuran (Tinggi, Lebar, Channel): {img_bgr.shape}
```

Gambar berhasil dimuat, ukuran (Tinggi, Lebar, Channel): (1052, 780, 3)

Cropping Manual Wajah dan Background

```
In [5]: print("Pilih ROI untuk Wajah, lalu tekan ENTER atau SPACE...")
r_wajah = cv2.selectROI("Pilih Wajah", img_bgr, fromCenter=False, showCrosshair=True)

print("Pilih ROI untuk Background, lalu tekan ENTER atau SPACE...")
r_bg = cv2.selectROI("Pilih Background", img_bgr, fromCenter=False, showCrosshair=True)

cv2.destroyAllWindows()

x_w, y_w, w_w, h_w = r_wajah
crop_wajah_bgr = img_bgr[y_w:y_w+h_w, x_w:x_w+w_w]

x_b, y_b, w_b, h_b = r_bg
crop_bg_bgr = img_bgr[y_b:y_b+h_b, x_b:x_b+w_b]

print(f"Koordinat Wajah (x,y,w,h): {r_wajah}")
print(f"Koordinat Background (x,y,w,h): {r_bg}")
```

Pilih ROI untuk Wajah, lalu tekan ENTER atau SPACE...

Pilih ROI untuk Background, lalu tekan ENTER atau SPACE...

Koordinat Wajah (x,y,w,h): (86, 154, 586, 758)

Koordinat Background (x,y,w,h): (98, 199, 561, 693)

Resize 920x920 piksel

```
In [6]: target_size = (920, 920)
        interpolation = cv2.INTER_AREA

        wajah_resized_bgr = cv2.resize(crop_wajah_bgr, target_size, interpolation=interpolation)
        bg_resized_bgr = cv2.resize(crop_bg_bgr, target_size, interpolation=interpolation)
```

Menambahkan Anotasi

```
In [7]: wajah_annotated_bgr = wajah_resized_bgr.copy()
        font = cv2.FONT_HERSHEY_SIMPLEX
        posisi = (150, 80)
        ukuran_font = 1.5
        warna = (0, 0, 0)
        ketebalan = 3

        cv2.putText(wajah_annotated_bgr, 'Rizki Alfariz Ramadhan', posisi, font, ukuran_font, warna, ketebalan)

        path_simpan_annotated = os.path.join(RESULTS_DIR, 'wajah_annotated.jpg')
        cv2.imwrite(path_simpan_annotated, wajah_annotated_bgr)
```

Out[7]: True

Konversi Warna dan Simpan (wajah)

```
In [8]: wajah_rgb = cv2.cvtColor(wajah_annotated_bgr, cv2.COLOR_BGR2RGB)
        wajah_gray = cv2.cvtColor(wajah_annotated_bgr, cv2.COLOR_BGR2GRAY)
        wajah_hsv = cv2.cvtColor(wajah_annotated_bgr, cv2.COLOR_BGR2HSV)

        cv2.imwrite(os.path.join(RESULTS_DIR, 'wajah_gray.jpg'), wajah_gray)
        cv2.imwrite(os.path.join(RESULTS_DIR, 'wajah_hsv_visual.jpg'), wajah_hsv)
```

Out[8]: True

Konversi Warna dan Simpan (Background)

```
In [9]: bg_rgb = cv2.cvtColor(bg_resized_bgr, cv2.COLOR_BGR2RGB)
        bg_gray = cv2.cvtColor(bg_resized_bgr, cv2.COLOR_BGR2GRAY)
        bg_hsv = cv2.cvtColor(bg_resized_bgr, cv2.COLOR_BGR2HSV)

        cv2.imwrite(os.path.join(RESULTS_DIR, 'bg_rgb.jpg'), bg_resized_bgr)
        cv2.imwrite(os.path.join(RESULTS_DIR, 'bg_gray.jpg'), bg_gray)
        cv2.imwrite(os.path.join(RESULTS_DIR, 'bg_hsv_visual.jpg'), bg_hsv)
```

Out[9]: True

Menampilkan hasil

```
In [10]: fig, axes = plt.subplots(1, 3, figsize=(18, 6))

        axes[0].imshow(wajah_rgb)
        axes[0].set_title('Wajah Crop (RGB)')
        axes[0].axis('off')
```

```

axes[1].imshow(wajah_gray, cmap='gray')
axes[1].set_title('Wajah Crop (Grayscale)')
axes[1].axis('off')

axes[2].imshow(wajah_hsv)
axes[2].set_title('Wajah Crop (HSV)')
axes[2].axis('off')

plt.suptitle('Analisis Gambar Wajah', fontsize=16)
plt.show()

```

Analisis Gambar Wajah



Penjelasan

Pada pengerjaan soal, gambar selfie.jpg diproses melalui beberapa tahap. Pertama, dilakukan cropping manual untuk mengambil dua ROI: bagian wajah sebagai fokus utama dan bagian latar belakang. Cropping membantu mengisolasi elemen lain yang ingin diamati secara terpisah. Kedua ROI menjadi ukuran yang sama (920×920 piksel), dan pada gambar wajah ditambahkan anotasi teks berupa nama di bagian atas.

Tahap berikutnya adalah konversi warna. Kedua gambar hasil crop diubah ke Grayscale dan HSV. Pada Grayscale, seluruh informasi warna dihilangkan sehingga gambar hanya menampilkan tingkat kecerahan (gelap–terang). Sebaliknya, konversi ke HSV memecah warna menjadi tiga komponen: Hue (rona), Saturation (kejenuhan), dan Value (kecerahan). Visualisasi HSV sering tampak berbeda dari gambar asli karena informasi warna direpresentasikan ulang. Representasi ini sangat berguna dalam analisis berbasis warna, seperti segmentasi objek, karena HSV lebih stabil terhadap variasi cahaya.

Secara keseluruhan, proses cropping dan konversi warna ini menunjukkan bagaimana area penting dapat diisolasi, serta bagaimana gambar dapat direpresentasikan dalam bentuk berbeda untuk mendukung berbagai kebutuhan analisis citra.

Soal 2 — Manipulasi Channel Warna RGB

- Gunakan gambar hasil crop dari Soal 1.

- Konversikan gambar ke ruang warna **RGB**.
- Lakukan manipulasi channel warna dengan cara:
 - **Naikkan intensitas channel merah sebanyak 50 poin** (maksimum 255).
 - **Turunkan intensitas channel biru sebanyak 30 poin** (minimum 0).
- Teknik atau cara menaikkan/menurunkan intensitas **dibebaskan**, asalkan logis dan hasilnya terlihat.
- Gabungkan kembali channel warna dan **simpan gambar hasil modifikasi dalam format .png**.
- **Tampilkan histogram per channel (R, G, B)** untuk gambar asli dan hasil modifikasi menggunakan `matplotlib.pyplot.hist`.
- Jelaskan dampak perubahan RGB pada warna gambar dalam sel **Markdown**.

Load Image Soal 1

```
In [11]: path_wajah_annotated = os.path.join(RESULTS_DIR, 'wajah_annotated.jpg')
img_wajah_bgr = cv2.imread(path_wajah_annotated)

if img_wajah_bgr is None:
    print(f"Error: Gagal memuat gambar 'wajah_annotated.jpg' dari {RESULTS_DIR}")
else:
    print("Gambar 'wajah_annotated.jpg' berhasil dimuat.")
```

Gambar 'wajah_annotated.jpg' berhasil dimuat.

Konversi ke RGB dan Split Channels

```
In [12]: img_wajah_rgb = cv2.cvtColor(img_wajah_bgr, cv2.COLOR_BGR2RGB)
r_channel, g_channel, b_channel = cv2.split(img_wajah_rgb)
```

Manipulasi Channel (Red +50, Blue -30)

```
In [13]: add_value = np.full(r_channel.shape, 50, dtype=np.uint8)
sub_value = np.full(b_channel.shape, 30, dtype=np.uint8)

r_channel_mod = cv2.add(r_channel, add_value)
b_channel_mod = cv2.subtract(b_channel, sub_value)
```

Gabungkan Image dan Simpan

```
In [14]: img_modified_rgb = cv2.merge([r_channel_mod, g_channel, b_channel_mod])

path_simpan_mod = os.path.join(RESULTS_DIR, 'wajah_rgb_modified.png')

img_modified_bgr = cv2.cvtColor(img_modified_rgb, cv2.COLOR_RGB2BGR)
cv2.imwrite(path_simpan_mod, img_modified_bgr)
```

```
print(f"Gambar modifikasi disimpan di: {path_simpan_mod}")
```

Gambar modifikasi disimpan di: e:\KIKI\MULMED\Sistem-Teknologi-Multimedia\Worksheet4\results_ws4\wajah_rgb_modified.png

Perbandingan Hasil Manipulasi

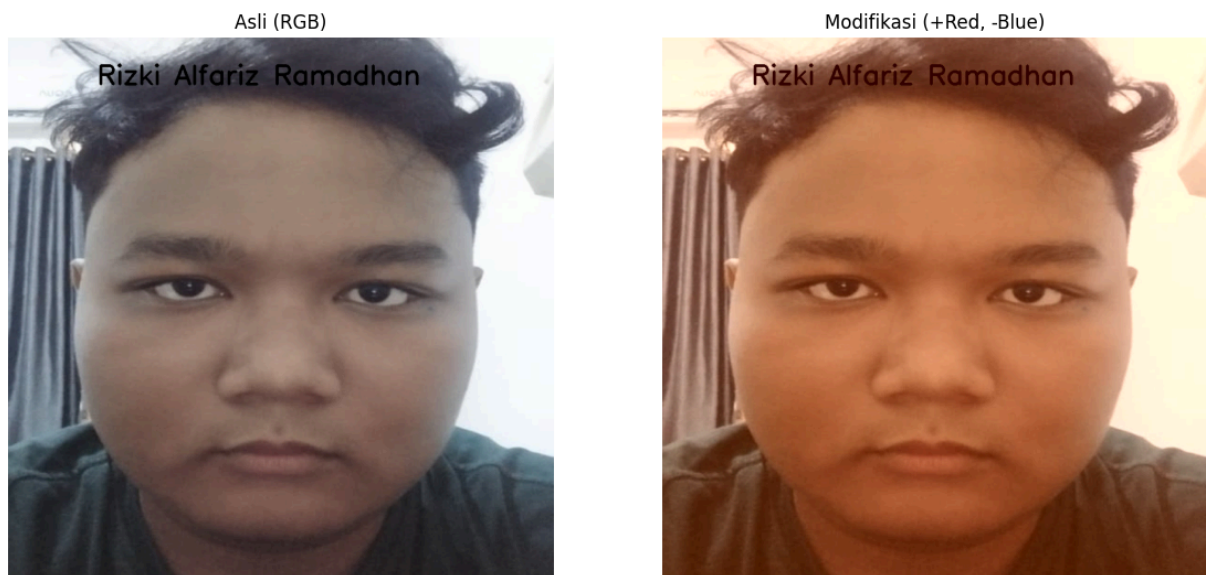
```
In [15]: fig, axes = plt.subplots(1, 2, figsize=(14, 7))

axes[0].imshow(img_wajah_rgb)
axes[0].set_title('Asli (RGB)')
axes[0].axis('off')

axes[1].imshow(img_modified_rgb)
axes[1].set_title('Modifikasi (+Red, -Blue)')
axes[1].axis('off')

plt.suptitle('Perbandingan Visual Manipulasi Channel', fontsize=16)
plt.show()
```

Perbandingan Visual Manipulasi Channel



Perbandingan Channel Asli dan Manipulasi

```
In [16]: fig, axes = plt.subplots(2, 3, figsize=(18, 10))
fig.suptitle('Perbandingan Histogram (Asli vs. Modifikasi)', fontsize=16)

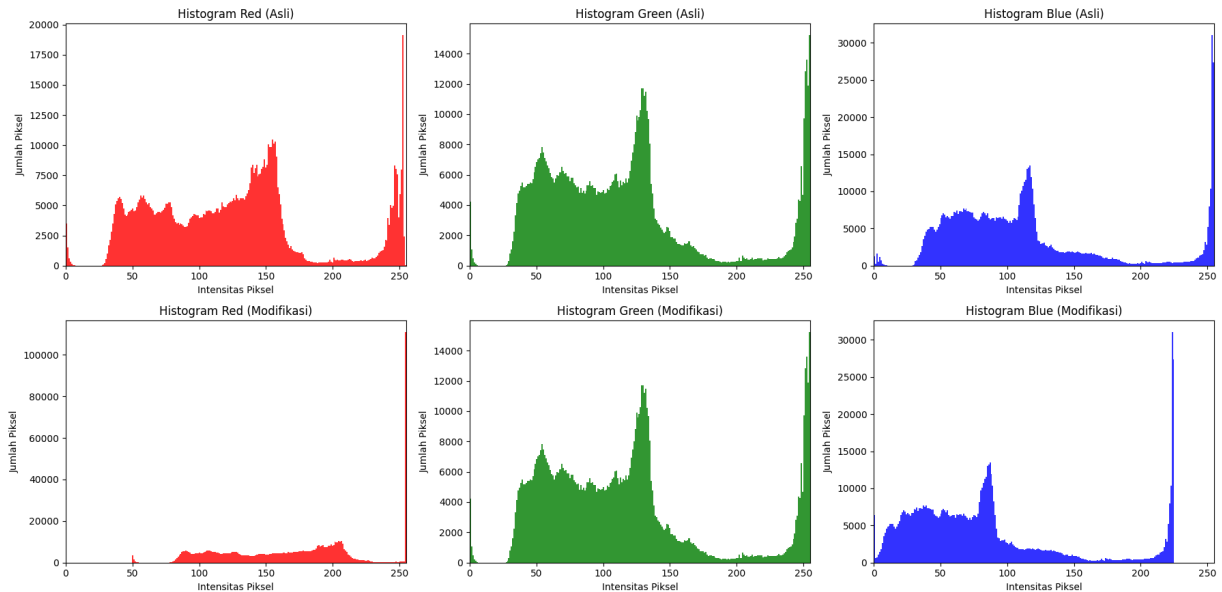
channels = [('Asli', [r_channel, g_channel, b_channel]),
            ('Modifikasi', [r_channel_mod, g_channel, b_channel_mod])]
colors = ['red', 'green', 'blue']
names = ['Red', 'Green', 'Blue']

for row, (label, chs) in enumerate(channels):
    for col, (ch, color, name) in enumerate(zip(chs, colors, names)):
        ax = axes[row, col]
        ax.hist(ch.flatten(), bins=256, range=(0, 255), color=color, alpha=0.8)
```

```
ax.set_title(f"Histogram {name} ({label})")
ax.set_xlim([0, 255])
ax.set_xlabel('Intensitas Pixel')
ax.set_ylabel('Jumlah Pixel')

plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()
```

Perbandingan Histogram (Asli vs. Modifikasi)



Penjelasan

Setelah menaikkan intensitas channel merah sebesar 50 poin dan menurunkan channel biru sebesar 30 poin, tampilan gambar berubah menjadi lebih "hangat", seperti yang ditunjukkan pada gambar perbandingan visual. Warna kulit yang semula natural kini terlihat jauh lebih oranye. Efek ini terjadi karena komponen merah menjadi lebih dominan di seluruh gambar, sementara nuansa kebiruan pada bayangan dan area netral telah berkurang secara signifikan.

Pergeseran warna ini selaras dengan data histogram yang ditampilkan. Pada channel merah (plot kiri), distribusi piksel pada gambar modifikasi terlihat jelas bergeser ke kanan (menuju nilai intensitas yang lebih tinggi) dibandingkan gambar aslinya. Sebaliknya, channel biru (plot kanan) menunjukkan pergeseran yang jelas ke kiri (menuju nilai yang lebih rendah). Sementara itu, channel hijau (plot tengah) terlihat stabil dan identik antara gambar asli dan modifikasi, yang mengkonfirmasi bahwa channel tersebut tidak dimodifikasi.

Soal 3 — Deteksi Tepi dan Filter Citra

- Ambil gambar **objek dengan background bertekstur** (misalnya kain bermotif, jerami, atau batu).
- Terapkan **edge detection (Canny)** dan tampilkan hasilnya.

- Lakukan **thresholding dengan nilai ambang tertentu** (bebas Anda tentukan) agar hanya objek utama yang tersisa.
- Buat **bounding box** di sekitar objek hasil segmentasi (boleh manual atau otomatis).
- Terapkan **filter blur** dan **filter sharpening**, lalu **bandingkan hasil keduanya**.
- Jelaskan bagaimana setiap filter memengaruhi detail gambar dalam format **Markdown**.

Path Image

```
In [17]: BASE_DIR = os.getcwd()
ASSETS_DIR = os.path.join(BASE_DIR, 'assets_ws4')
RESULTS_DIR = os.path.join(BASE_DIR, 'results_ws4')

if not os.path.exists(RESULTS_DIR):
    os.makedirs(RESULTS_DIR)

NEW_IMAGE_NAME = 'objek.jpg'
NEW_IMAGE_PATH = os.path.join(ASSETS_DIR, NEW_IMAGE_NAME)
```

Memuat Gambar dan Konversi Grayscale

```
In [18]: img_obj_bgr = cv2.imread(NEW_IMAGE_PATH)

if img_obj_bgr is None:
    print(f"Error: Gagal memuat gambar dari {NEW_IMAGE_PATH}")
    print("Pastikan nama file di Cell 3 sudah benar dan file ada di folder 'assets_")
else:
    print(f"Gambar '{NEW_IMAGE_NAME}' berhasil dimuat.")
    img_obj_gray = cv2.cvtColor(img_obj_bgr, cv2.COLOR_BGR2GRAY)
```

Gambar 'objek.jpg' berhasil dimuat.

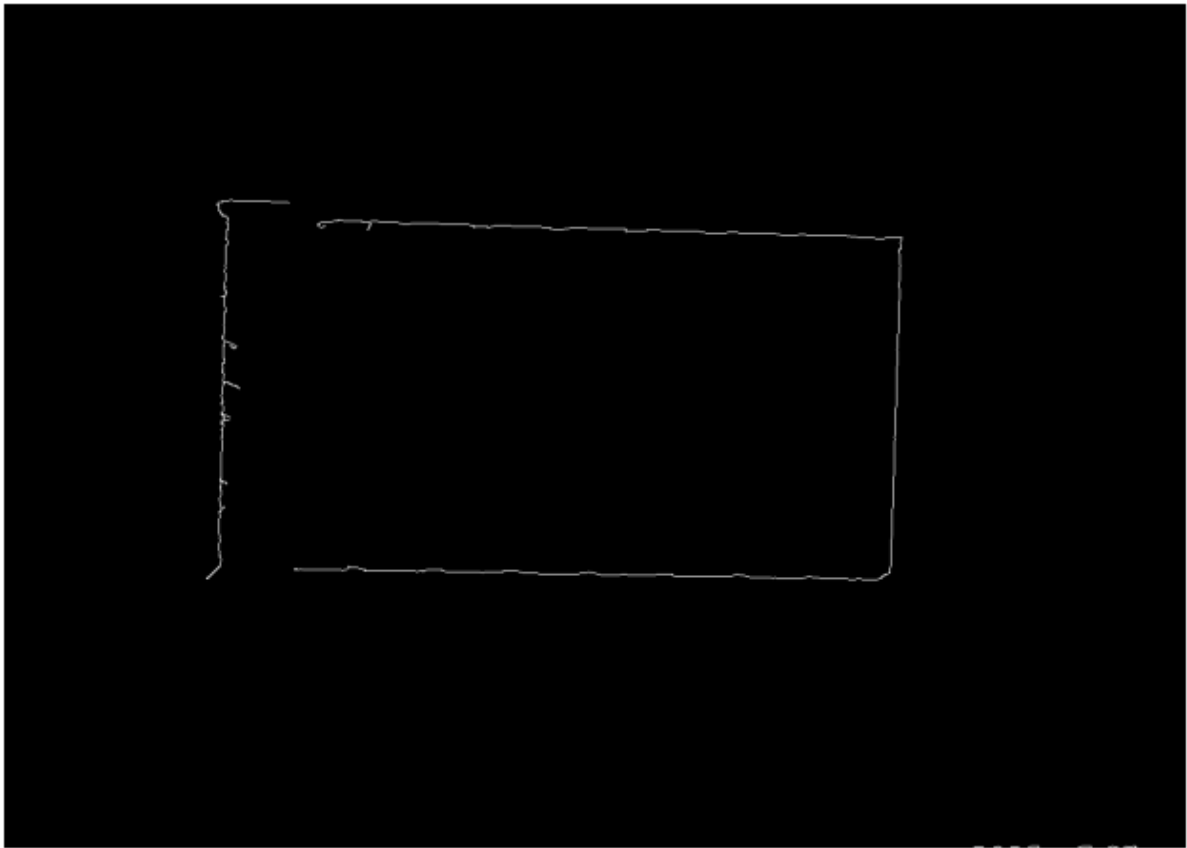
Deteksi Tepi (Canny)

```
In [19]: edges = cv2.Canny(img_obj_gray, 100, 500)

path_simpan_canny = os.path.join(RESULTS_DIR, 'objek_canny_edges.jpg')
cv2.imwrite(path_simpan_canny, edges)

plt.figure(figsize=(8, 8))
plt.imshow(edges, cmap='gray')
plt.title('Deteksi Tepi Canny')
plt.axis('off')
plt.show()
```

Deteksi Tepi Canny



Thresholding

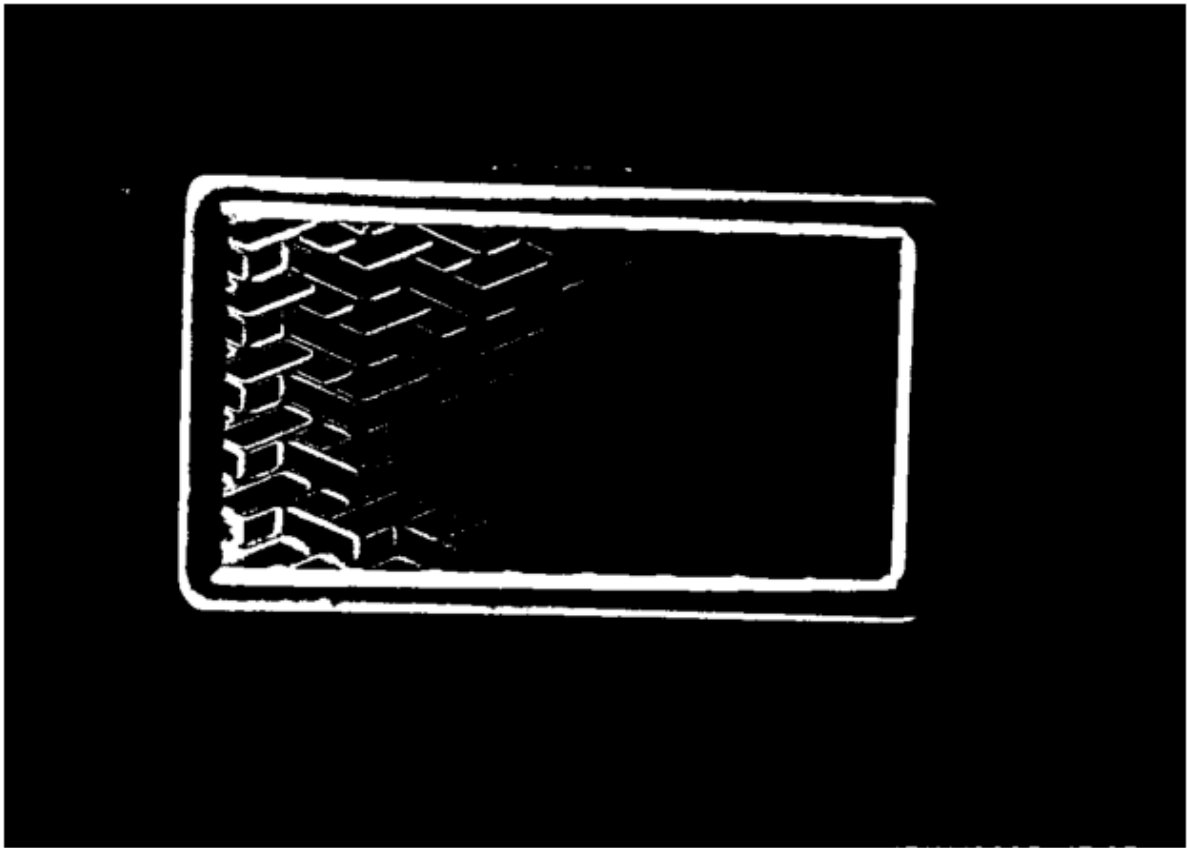
```
In [20]: THRESH_VALUE = 180

ret, thresh_img = cv2.threshold(img_obj_gray, THRESH_VALUE, 255, cv2.THRESH_BINARY)

path_simpan_thresh = os.path.join(RESULTS_DIR, 'objek_thresholdded.jpg')
cv2.imwrite(path_simpan_thresh, thresh_img)

plt.figure(figsize=(8, 8))
plt.imshow(thresh_img, cmap='gray')
plt.title(f'Hasil Thresholding (Nilai = {THRESH_VALUE})')
plt.axis('off')
plt.show()
```

Hasil Thresholding (Nilai = 180)



Bounding Box Otomatis

```
In [21]: contours, hierarchy = cv2.findContours(thresh_img, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

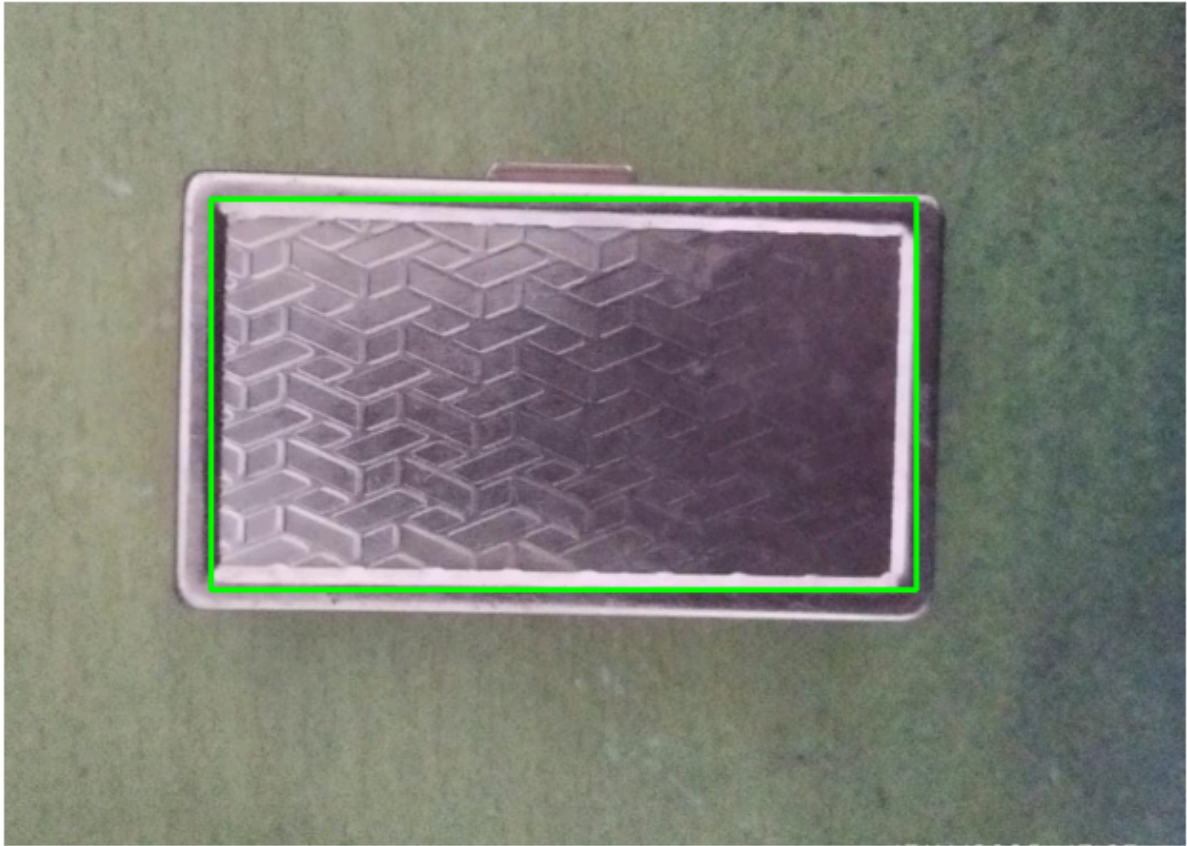
img_bbox_bgr = img_obj_bgr.copy()

if len(contours) > 0:
    c = max(contours, key=cv2.contourArea)
    x, y, w, h = cv2.boundingRect(c)
    cv2.rectangle(img_bbox_bgr, (x, y), (x+w, y+h), (0, 255, 0), 3)
else:
    print("Tidak ada kontur yang ditemukan untuk Bounding Box.")

path_simpan_bbox = os.path.join(RESULTS_DIR, 'objek_bounding_box_otomatis.jpg')
cv2.imwrite(path_simpan_bbox, img_bbox_bgr)

plt.figure(figsize=(8, 8))
plt.imshow(cv2.cvtColor(img_bbox_bgr, cv2.COLOR_BGR2RGB))
plt.title('Bounding Box pada objek')
plt.axis('off')
plt.show()
```

Bounding Box pada objek



Filter Blur (Gaussian)

```
In [22]: blur_kernel_size = (15, 15)
img_blur = cv2.GaussianBlur(img_obj_bgr, blur_kernel_size, 0)

path_simpan_blur = os.path.join(RESULTS_DIR, 'objek_blurred.jpg')
cv2.imwrite(path_simpan_blur, img_blur)
```

Out[22]: True

Filter Sharpening

```
In [23]: kernel_sharpen = np.array([
    [-1, -1, -1],
    [-1,  9, -1],
    [-1, -1, -1]
])

img_sharpen = cv2.filter2D(img_obj_bgr, -1, kernel_sharpen)

path_simpan_sharpen = os.path.join(RESULTS_DIR, 'objek_sharpened.jpg')
cv2.imwrite(path_simpan_sharpen, img_sharpen)
```

Out[23]: True

Visualisasi Perbandingan Filter

```
In [24]: # Konversi BGR ke RGB untuk gambar yang belum dikonversi
img_obj_rgb = cv2.cvtColor(img_obj_bgr, cv2.COLOR_BGR2RGB)
img_bbox_rgb = cv2.cvtColor(img_bbox_bgr, cv2.COLOR_BGR2RGB)
img_blur_rgb = cv2.cvtColor(img_blur, cv2.COLOR_BGR2RGB)
img_sharpen_rgb = cv2.cvtColor(img_sharpen, cv2.COLOR_BGR2RGB)

# Buat grid plot 2x3
fig, axes = plt.subplots(2, 3, figsize=(24, 16))
fig.suptitle('Visualisasi Semua Tahapan Pemrosesan (Soal 3)', fontsize=20)

# Baris 1
axes[0, 0].imshow(img_obj_rgb)
axes[0, 0].set_title('Gambar Asli (Objek)')

axes[0, 1].imshow(edges, cmap='gray')
axes[0, 1].set_title('Deteksi Tepi Canny (objek_canny_edges.jpg)')

axes[0, 2].imshow(thresh_img, cmap='gray')
axes[0, 2].set_title(f'Thresholding (objek_thresholded.jpg)')

# Baris 2
axes[1, 0].imshow(img_bbox_rgb)
axes[1, 0].set_title('Bounding Box (objek_bounding_box.jpg)')

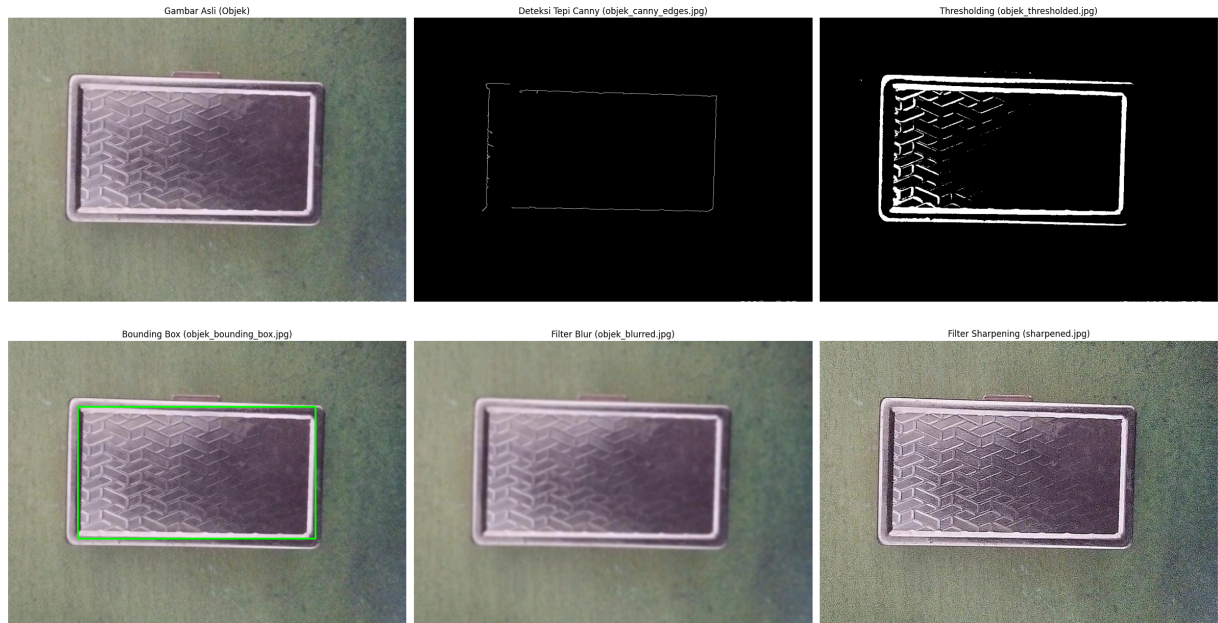
axes[1, 1].imshow(img_blur_rgb)
axes[1, 1].set_title(f'Filter Blur (objek_blurred.jpg)')

axes[1, 2].imshow(img_sharpen_rgb)
axes[1, 2].set_title('Filter Sharpening (sharpened.jpg)')

for ax in axes.flat:
    ax.axis('off')

plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()
```

Visualisasi Semua Tahapan Pemrosesan (Soal 3)



Penjelasan

Deteksi Tepi Canny seperti yang terlihat pada gambar "Deteksi Tepi Canny," filter ini menghilangkan semua informasi permukaan (warna dan tekstur) dan hanya menyisakan garis kontur terkuat. Canny hanya mendeteksi garis tepi luar (outline) dari objek dan mengabaikan pola di permukaannya. Ini menunjukkan bahwa perbedaan intensitas antara objek dan latar belakang hijau jauh lebih kuat daripada detail tekstur pada objek itu sendiri.

Threshold mengubah gambar menjadi hitam–putih. Metode ini berhasil memisahkan objek dari latar belakang karena perbedaan kecerahan jelas, dan sebagian pola ikut muncul karena memiliki intensitas di atas ambang.

Filter Blur (Gaussian) menghilangkan detail frekuensi tinggi (detail tajam). Seperti yang ditunjukkan pada "Filter Blur," gambar menjadi halus dan kabur. Efek ini paling jelas terlihat pada hilangnya ketajaman pola pada pemotong kuku dan hilangnya tekstur pada latar belakang hijau.

Filter Sharpening adalah kebalikan dari blur, filter sharpening menonjolkan detail frekuensi tinggi. Pada gambar "Filter Sharpening" perbedaannya sangat jelas, pola pada objek terlihat jauh lebih tajam dan jelas, dan tekstur "kasar" pada latar belakang hijau yang menjadi sangat menonjol.

Soal 4 — Deteksi Wajah dan Filter Digital Kreatif

- Ambil gambar diri Anda dengan ekspresi wajah **netral**.
- Lakukan **deteksi wajah dan landmark** menggunakan salah satu dari:

- **MediaPipe**, atau
- **Dlib**, atau
- **OpenCV**.
- Buat **overlay filter digital kreatif** karya Anda sendiri, misalnya:
 - topi, kumis, masker, helm, aksesoris, atau bentuk unik lainnya.
 - Filter boleh dibuat dari **gambar eksternal (PNG)** *atau* digambar langsung (misal bentuk lingkaran, garis, poligon, dll).
- Pastikan posisi overlay menyesuaikan **landmark wajah** dengan logis.
- **Gunakan alpha blending sebagai saran** agar hasil tampak lebih natural.
- Tampilkan perbandingan antara **gambar asli** dan **hasil dengan filter**.
- Jelaskan bagaimana Anda menghitung posisi overlay dan tantangan yang dihadapi selama implementasi (gunakan **Markdown**).

Import Library

```
In [25]: import cv2
import os
import matplotlib.pyplot as plt
import numpy as np
import mediapipe as mp
```

Inisialisasi MediaPipe

```
In [26]: BASE_DIR = os.getcwd()
ASSETS_DIR = os.path.join(BASE_DIR, 'assets_ws4')
RESULTS_DIR = os.path.join(BASE_DIR, 'results_ws4')

SELFIE_PATH = os.path.join(ASSETS_DIR, 'selfie.jpg')
OVERLAY_PATH = os.path.join(ASSETS_DIR, 'kacamata.png')

mp_face_mesh = mp.solutions.face_mesh
face_mesh = mp_face_mesh.FaceMesh(
    static_image_mode=True,
    max_num_faces=1,
    refine_landmarks=True,
    min_detection_confidence=0.5)
```

Load Image

```
In [27]: img_selfie_bgr = cv2.imread(SELFIE_PATH)
img_overlay_rgba = cv2.imread(OVERLAY_PATH, cv2.IMREAD_UNCHANGED)

if img_selfie_bgr is None:
    print(f"Error: Gagal memuat {SELFIE_PATH}")
elif img_overlay_rgba is None:
    print(f"Error: Gagal memuat {OVERLAY_PATH}. Pastikan file ada di assets_ws4.")
```

```
else:
    print("Selfie dan Kacamata berhasil dimuat.")
```

Selfie dan Kacamata berhasil dimuat.

Deteksi Landmark

```
In [28]: mp_drawing = mp.solutions.drawing_utils
mp_drawing_styles = mp.solutions.drawing_styles
mp_face_mesh = mp.solutions.face_mesh
img_filtered = img_selfie_bgr.copy()
h_img, w_img = img_selfie_bgr.shape[:2]

img_selfie_rgb_for_mesh = cv2.cvtColor(img_selfie_bgr, cv2.COLOR_BGR2RGB)
results = face_mesh.process(img_selfie_rgb_for_mesh)

x_pos = 0
y_pos = 0
new_height = 0
new_width = 0

# Pengaturan size
SKALA_KACAMATA = 2.0

if results.multi_face_landmarks:
    landmarks = results.multi_face_landmarks[0].landmark

    lm_left_eye = landmarks[33]
    lm_right_eye = landmarks[263]

    x_left = int(lm_left_eye.x * w_img)
    y_left = int(lm_left_eye.y * h_img)
    x_right = int(lm_right_eye.x * w_img)
    y_right = int(lm_right_eye.y * h_img)

    # 1. Hitung lebar kacamata
    width_asli = x_right - x_left
    glasses_width = int(width_asli * SKALA_KACAMATA)

    h_ov, w_ov = img_overlay_rgba.shape[:2]
    aspect_ratio = h_ov / w_ov
    new_height = int(glasses_width * aspect_ratio)
    new_width = glasses_width

    # 2. Resize kacamata
    resized_overlay_rgba = cv2.resize(img_overlay_rgba, (new_width, new_height))

    # 3. Tentukan posisi top-left (x, y) kacamata
    selisih_lebar = new_width - width_asli
    x_pos = x_left - (selisih_lebar // 2)

    y_pos = int((y_left + y_right) / 2) - (new_height // 2)

    print("Landmark terdeteksi dan posisi kalkulasi selesai.")
    print(f"Ukuran kacamata diubah ke: {new_width} x {new_height} piksel")
else:
```

```
print("Tidak ada wajah yang terdeteksi.")
```

Landmark terdeteksi dan posisi kalkulasi selesai.

Ukuran kacamata diubah ke: 574 x 574 piksel

```
In [29]: img_landmark = img_selfie_bgr.copy()
         h_img, w_img = img_landmark.shape[:2]

         if results.multi_face_landmarks:
             for face_landmarks in results.multi_face_landmarks:
                 idx_left_eye = 33
                 idx_right_eye = 263

                 x_left = int(face_landmarks.landmark[idx_left_eye].x * w_img)
                 y_left = int(face_landmarks.landmark[idx_left_eye].y * h_img)
                 x_right = int(face_landmarks.landmark[idx_right_eye].x * w_img)
                 y_right = int(face_landmarks.landmark[idx_right_eye].y * h_img)

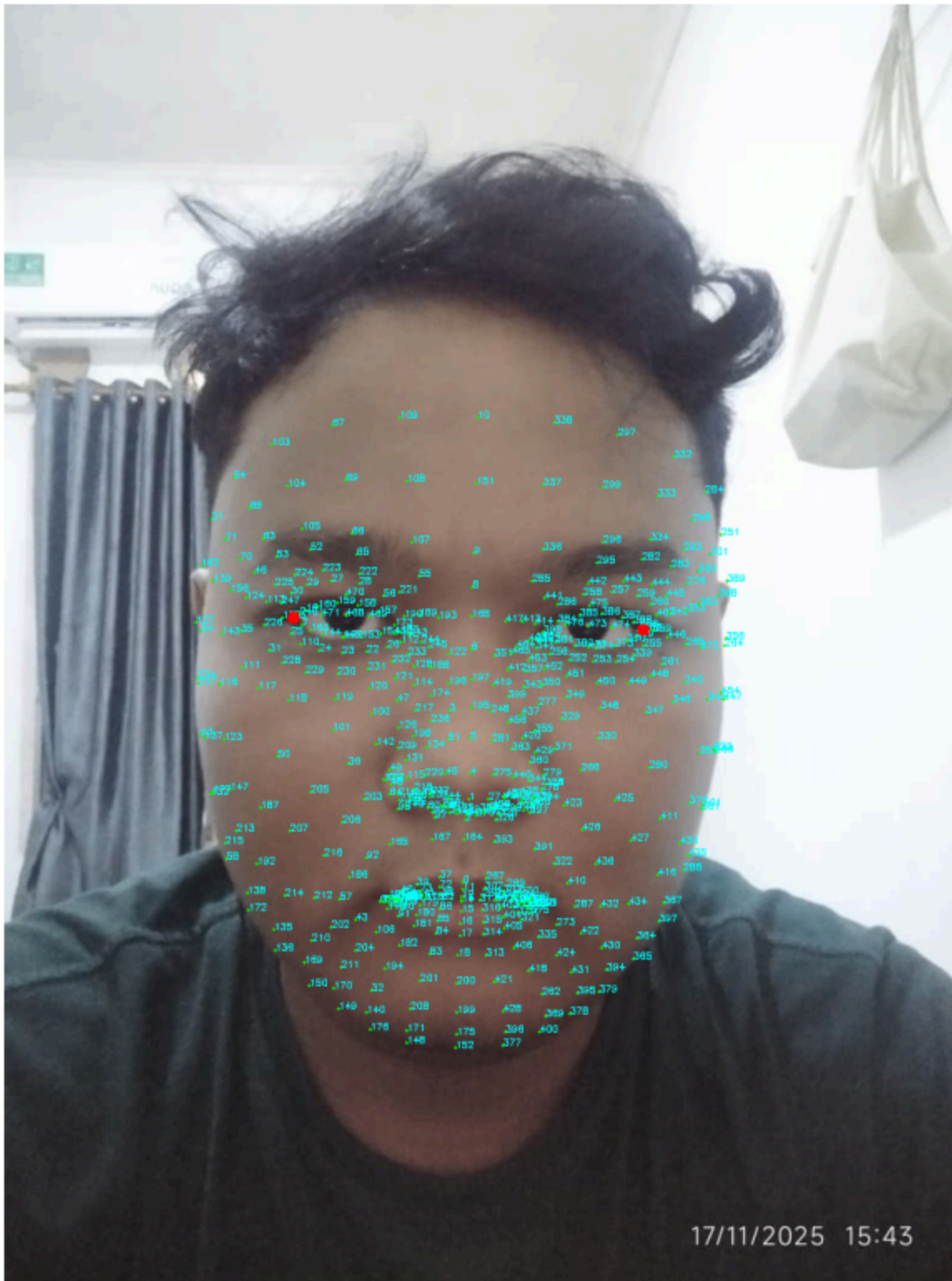
                 for idx, lm in enumerate(face_landmarks.landmark):
                     x = int(lm.x * w_img)
                     y = int(lm.y * h_img)
                     cv2.circle(img_landmark, (x, y), 1, (0, 255, 0), -1)
                     cv2.putText(img_landmark, str(idx), (x, y),
                                cv2.FONT_HERSHEY_SIMPLEX,
                                0.25,
                                (255, 255, 0),
                                1)

                     cv2.circle(img_landmark, (x_left, y_left), 5, (0, 0, 255), -1)
                     cv2.circle(img_landmark, (x_right, y_right), 5, (0, 0, 255), -1)
             else:
                 print("Tidak ada landmark untuk digambar.")

         img_landmark_rgb = cv2.cvtColor(img_landmark, cv2.COLOR_BGR2RGB)
         plt.figure(figsize=(10, 10))
         plt.imshow(img_landmark_rgb)
         plt.title("Visualisasi Landmark")
         plt.axis('off')
         plt.show()

         path_simpan_landmark = os.path.join(RESULTS_DIR, 'selfie_landmark_numbers.jpg')
         cv2.imwrite(path_simpan_landmark, img_landmark)
```

Visualisasi Landmark



Out[29]: True

Logika Overlay (Alpha Blending Inline)

```
In [30]: if results.multi_face_landmarks:
# Tentukan Region of Interest (ROI) di gambar selfie
y1, y2 = max(0, y_pos), min(y_pos + new_height, h_img)
```

```

x1, x2 = max(0, x_pos), min(x_pos + new_width, w_img)

# Sesuaikan ukuran overlay jika terpotong oleh batas gambar
h_roi = y2 - y1
w_roi = x2 - x1
resized_overlay_rgba_cut = resized_overlay_rgba[0:h_roi, 0:w_roi]

# Pisahkan kaca mata (RGB) dan mask (Alpha)
overlay_rgb = resized_overlay_rgba_cut[:, :, :3]
alpha_mask = resized_overlay_rgba_cut[:, :, 3] / 255.0

# Buat inverse mask
inverse_alpha_mask = 1.0 - alpha_mask

# Ambil area ROI dari selfie
roi = img_filtered[y1:y2, x1:x2]

# 'Gelapkan' area di selfie tempat kaca mata akan diletakkan
roi_bg = (inverse_alpha_mask[..., np.newaxis] * roi).astype(roi.dtype)

# Isolasi kaca mata (tanpa background)
roi_fg = (alpha_mask[..., np.newaxis] * overlay_rgb).astype(roi.dtype)

# Gabungkan kaca mata dan background
blended_roi = cv2.add(roi_bg, roi_fg)

# Tempelkan kembali ROI yang sudah di-blend ke gambar utama
img_filtered[y1:y2, x1:x2] = blended_roi

print("Filter kaca mata berhasil diterapkan (inline).")

# Simpan hasil
path_simpan_filter = os.path.join(RESULTS_DIR, 'selfie_kacamata.jpg')
cv2.imwrite(path_simpan_filter, img_filtered)

```

Filter kaca mata berhasil diterapkan (inline).

Out[30]: True

Visualisasi Perbandingan

```

In [31]: img_selfie_rgb = cv2.cvtColor(img_selfie_bgr, cv2.COLOR_BGR2RGB)
img_filtered_rgb = cv2.cvtColor(img_filtered, cv2.COLOR_BGR2RGB)

fig, axes = plt.subplots(1, 2, figsize=(16, 8))

axes[0].imshow(img_selfie_rgb)
axes[0].set_title('Gambar Asli (Netral)')
axes[0].axis('off')

axes[1].imshow(img_filtered_rgb)
axes[1].set_title('Hasil Filter Kacamata (MediaPipe)')
axes[1].axis('off')

plt.suptitle('Perbandingan Filter Digital Kreatif', fontsize=16)
plt.show()

```

Perbandingan Filter Digital Kreatif

Gambar Asli (Netral)



Hasil Filter Kacamata (MediaPipe)



Penjelasan

Kalkulasi posisi overlay kacamata bergantung penuh pada MediaPipe Face Mesh untuk penempatan yang proporsional. Saya berfokus pada dua landmark krusial, Landmark 33 (sudut mata kiri luar) dan Landmark 263 (sudut mata kanan luar), untuk menghitung lebar dasar kacamata. Lebar ini kemudian dikalikan dengan variabel `SKALA_KACAMATA` (misalnya 2) untuk memberikan kontrol manual atas ukurannya, sementara tinggi kacamata disesuaikan secara otomatis menggunakan aspect ratio asli dari file kacamata.png. Posisi vertikal (`y_pos`) diatur agar kacamata berada di tengah-tengah kedua mata, dan posisi horizontal (`x_pos`) disesuaikan secara dinamis agar kacamata yang sudah diskalakan tetap terpusat di wajah. Tantangan yang dihadapi adalah penyesuaian size dan skala overlay, penentuan titik landmark, dan alpha blending

Soal 5 — Perspektif dan Peningkatan Kualitas Citra

- Ambil gambar **objek datar** seperti karya tangan di kertas, tulisan di papan tulis, atau foto produk di meja dengan kondisi pencahayaan atau sudut yang tidak ideal.
- Lakukan **preprocessing** untuk memperbaiki tampilan agar lebih rapi dan jelas, dengan langkah-langkah:
 - Konversi ke **grayscale**.
 - **Koreksi perspektif (transformasi homografi)** menggunakan **4 titik manual** agar objek terlihat sejajar dan tidak terdistorsi.
 - Terapkan **thresholding adaptif atau Otsu** (pilih salah satu, dan jelaskan alasan pilihan Anda).

- Tampilkan **setiap tahap pemrosesan dalam satu grid** agar mudah dibandingkan.
- Jelaskan fungsi masing-masing tahap dan bagaimana teknik ini meningkatkan kualitas visual citra (gunakan **Markdown**).

Paths

```
In [32]: BASE_DIR = os.getcwd()
ASSETS_DIR = os.path.join(BASE_DIR, 'assets_ws4')
RESULTS_DIR = os.path.join(BASE_DIR, 'results_ws4')

if not os.path.exists(RESULTS_DIR):
    os.makedirs(RESULTS_DIR)
```

Load Gambar

```
In [33]: DOC_IMAGE_NAME = 'paper.jpg'
DOC_IMAGE_PATH = os.path.join(ASSETS_DIR, DOC_IMAGE_NAME)

img_doc_bgr = cv2.imread(DOC_IMAGE_PATH)

if img_doc_bgr is None:
    print(f"Error: Gagal memuat gambar dari {DOC_IMAGE_PATH}")
    print("Pastikan nama file di Cell 3 sudah benar dan file ada di folder 'assets_'
else:
    print(f"Gambar '{DOC_IMAGE_NAME}' berhasil dimuat.")
```

Gambar 'paper.jpg' berhasil dimuat.

Konversi Grayscale

```
In [34]: img_gray = cv2.cvtColor(img_doc_bgr, cv2.COLOR_BGR2GRAY)

path_simpan_gray = os.path.join(RESULTS_DIR, 'paper_grayscale.jpg')
cv2.imwrite(path_simpan_gray, img_gray)

print("Gambar berhasil dikonversi ke grayscale.")
```

Gambar berhasil dikonversi ke grayscale.

Pencarian Titik Manual

```
In [35]: points_src_list = []
img_clone = img_doc_bgr.copy()

scale = 0.5
img_display = cv2.resize(img_clone, None, fx=scale, fy=scale)

def select_points(event, x, y, flags, param):
    global points_src_list

    x_orig = int(x / scale)
    y_orig = int(y / scale)
```

```

if event == cv2.EVENT_LBUTTONDOWN and len(points_src_list) < 4:
    points_src_list.append((x_orig, y_orig))
    cv2.circle(img_display, (x, y), 5, (0, 0, 255), -1)
    print(f"Titik {len(points_src_list)}: ({x_orig}, {y_orig})")

cv2.namedWindow("Pilih 4 Titik")
cv2.setMouseCallback("Pilih 4 Titik", select_points)

print("--- INSTRUKSI ---")
print("Klik 4 titik sudut dokumen: Kiri Atas → Kanan Atas → Kanan Bawah → Kiri Bawah")
print("Tekan 'q' untuk lanjut setelah memilih 4 titik.")

while True:
    cv2.imshow("Pilih 4 Titik", img_display)
    key = cv2.waitKey(1) & 0xFF
    if key == ord('q') and len(points_src_list) == 4:
        break

cv2.destroyAllWindows()

print("Titik akhir:", points_src_list)

```

--- INSTRUKSI ---

Klik 4 titik sudut dokumen: Kiri Atas → Kanan Atas → Kanan Bawah → Kiri Bawah.

Tekan 'q' untuk lanjut setelah memilih 4 titik.

Titik 1: (152, 140)

Titik 2: (784, 160)

Titik 3: (692, 1196)

Titik 4: (116, 1108)

Titik akhir: [(152, 140), (784, 160), (692, 1196), (116, 1108)]

Koreksi Perspektif

```

In [37]: if len(points_src_list) == 4:
    pts_src = np.float32(points_src_list)
    width_top = np.sqrt(((pts_src[1][0] - pts_src[0][0])**2) + ((pts_src[1][1] - pts_src[0][1])**2))
    width_bottom = np.sqrt(((pts_src[2][0] - pts_src[3][0])**2) + ((pts_src[2][1] - pts_src[3][1])**2))
    maxWidth = max(int(width_top), int(width_bottom))

    height_left = np.sqrt(((pts_src[3][0] - pts_src[0][0])**2) + ((pts_src[3][1] - pts_src[0][1])**2))
    height_right = np.sqrt(((pts_src[2][0] - pts_src[1][0])**2) + ((pts_src[2][1] - pts_src[1][1])**2))
    maxHeight = max(int(height_left), int(height_right))
    pts_dst = np.float32([
        [0, 0],
        [maxWidth - 1, 0],
        [maxWidth - 1, maxHeight - 1],
        [0, maxHeight - 1]
    ])

    matrix = cv2.getPerspectiveTransform(pts_src, pts_dst)

    img_warped = cv2.warpPerspective(img_gray, matrix, (maxWidth, maxHeight))

    path_simpan_warped = os.path.join(RESULTS_DIR, 'paper_koreksi.jpg')
    cv2.imwrite(path_simpan_warped, img_warped)

```

```
print("Koreksi perspektif berhasil.")
else:
    print("Error: 4 titik tidak dipilih. Jalankan ulang Cell 5.")
```

Koreksi perspektif berhasil.

Tresholidng (adaptif)

```
In [47]: img_thresh = cv2.adaptiveThreshold(
    img_warped, 255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    25,
    8
)

path_simpan_thresh = os.path.join(RESULTS_DIR, 'doc_step3_threshold.jpg')
cv2.imwrite(path_simpan_thresh, img_thresh)

print("Adaptive Thresholding berhasil diterapkan.")
```

Adaptive Thresholding berhasil diterapkan.

Visualisasi Perbandingan

```
In [48]: img_doc_rgb = cv2.cvtColor(img_doc_bgr, cv2.COLOR_BGR2RGB)

fig, axes = plt.subplots(2, 2, figsize=(16, 16))
fig.suptitle('Perbandingan Tahapan Pemrosesan (Soal 5)', fontsize=20)

axes[0, 0].imshow(img_doc_rgb)
axes[0, 0].set_title('Tahap 0: Gambar Asli (Miring)')

axes[0, 1].imshow(img_gray, cmap='gray')
axes[0, 1].set_title('Tahap 1: Grayscale')

axes[1, 0].imshow(img_warped, cmap='gray')
axes[1, 0].set_title('Tahap 2: Koreksi Perspektif (Warped)')

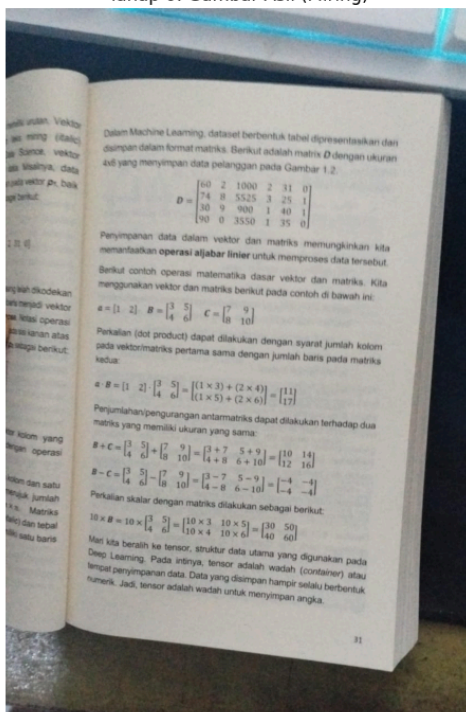
axes[1, 1].imshow(img_thresh, cmap='gray')
axes[1, 1].set_title('Tahap 3: Adaptive Thresholding (Hasil Akhir)')

for ax in axes.flat:
    ax.axis('off')

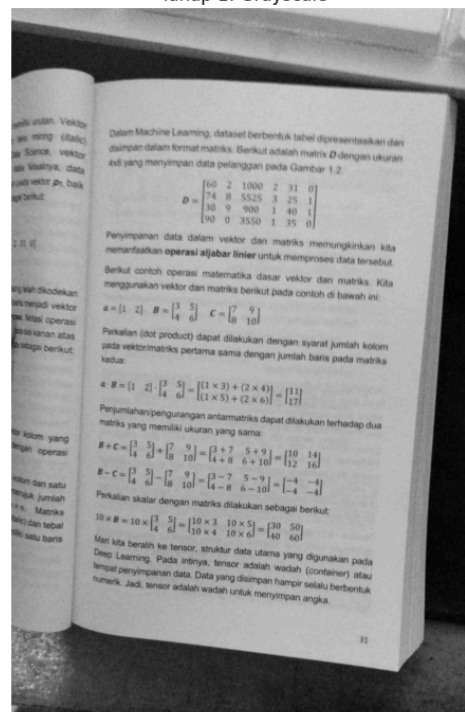
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()
```

Perbandingan Tahapan Pemrosesan (Soal 5)

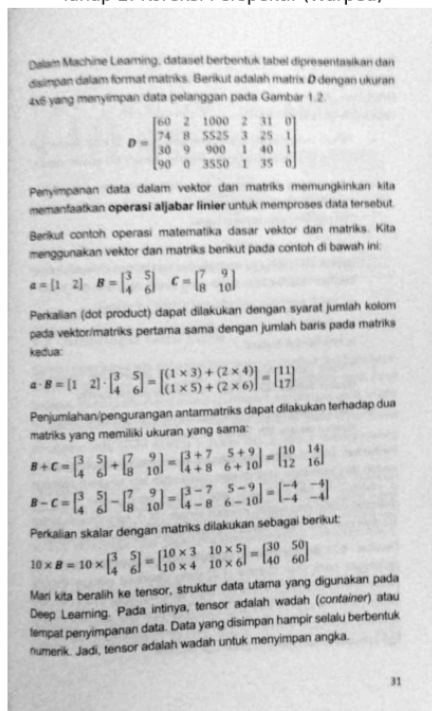
Tahap 0: Gambar Asli (Miring)



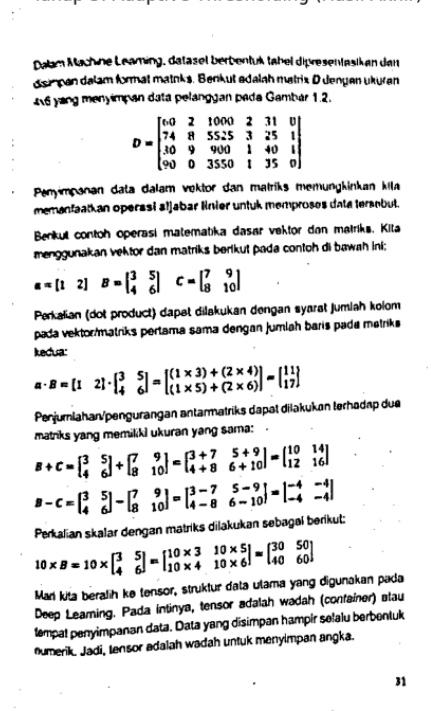
Tahap 1: Grayscale



Tahap 2: Koreksi Perspektif (Warped)



Tahap 3: Adaptive Thresholding (Hasil Akhir)



Penjelasan

Rangkaian pemrosesan ini bertujuan untuk mengubah gambar objek datar yang terdistorsi menjadi pindaian (scan) yang bersih dan lurus. Tahap pertama, konversi ke grayscale, menyederhanakan gambar menjadi 1 channel intensitas cahaya, yang menghilangkan gangguan warna dan mempercepat pemrosesan selanjutnya. Tahap kedua, koreksi

perspektif, adalah langkah paling penting dengan memilih 4 titik sudut objek yang miring dan memetakannya ke persegi panjang sempurna, distorsi geometris diperbaiki dan gambar "diluruskan" seolah dilihat dari atas. Terakhir, adaptive thresholding dipilih karena menangani pencahayaan tidak merata, seperti bayangan. Teknik ini menghitung nilai ambang secara lokal untuk setiap area kecil, secara efektif memaksa latar belakang (kertas) menjadi putih bersih dan tulisan (foreground) menjadi hitam pekat, sehingga menghasilkan citra biner yang sangat jelas dan mudah dibaca.

Aturan Umum Pengerjaan

- Kerjakan secara **mandiri**.
- Bantuan AI (seperti ChatGPT, Copilot, dsb.) diperbolehkan **dengan bukti percakapan** (screenshot / link / script percakapan).
- Source code antar mahasiswa harus berbeda.
- Jika mendapat bantuan teman, tuliskan nama dan NIM teman yang membantu.
- Plagiarisme akan dikenakan sanksi sesuai aturan akademik ITERA.
- Cantumkan seluruh **credit dan referensi** yang digunakan di bagian akhir notebook.
- Penjelasan setiap soal ditulis dalam **Markdown**, bukan di dalam komentar kode.

Aturan Pengumpulan

- Semua file kerja Anda (notebook `.ipynb`, gambar, dan hasil) **wajib diunggah ke GitHub repository tugas sebelumnya**.
 - Gunakan struktur folder berikut di dalam repo Anda:

```
/Nama_NIM_Repo/ # Nama repo sebelumnya
├── assets_ws4/    # berisi semua gambar atau video asli (input)
├── results_ws4/  # berisi semua hasil modifikasi dan output
├── worksheet4.ipynb
└── NIM_Worksheet4.pdf
```
- File yang dikumpulkan ke **Tally** hanya berupa **hasil PDF** dari notebook Anda, dengan format nama:
`NIM_Worksheet4.pdf`
- Pastikan notebook telah dijalankan penuh sebelum diekspor ke PDF.
- Sertakan tautan ke repository GitHub Anda di bagian atas notebook atau di halaman pertama PDF.



Catatan Akhir

Worksheet 4 ini bertujuan mengasah pemahaman Anda tentang manipulasi citra digital secara praktis. Gunakan kreativitas Anda untuk menghasilkan hasil

visual yang menarik dan penjelasan konseptual yang jelas.

Referensi

[Link ChatGPT](#)

[Link Github](#)